Trabajo práctico final

Qué escuchamos y cuándo: análisis de escuchas musicales

Tabla de contenidos

1 Introducción 1
2 Reglas generales 2
3 Excepción `DatoInvalidoError` (archivo `excepciones.py`) 3
4 Clase `Cancion` (archivo `cancion.py`) 4
4.1 Inicialización: `Cancion(titulo, artista, genero)` 4
4.2 Atributos y métodos 4
4.3 Ejemplos 4
5 Clase `Escucha` (archivo `escucha.py`) 5
5.1 Atributos de clase 5
5.2 Inicialización: `Escucha(cancion, dia, hora)` 5
5.3 Propiedades y métodos especiales 5
5.4 Ejemplos 6
6 Clase `Cola` (archivo `cola.py`) 7
6.1 Clase `Nodo` 7
6.2 Clase `Cola` 7
7 Clase `Historial` y función `en_franja` (archivo `historial.py`) 8
7.1 Datos de ejemplo 9
7.2 Métodos 9
7.3 Cómo funciona `tendencias` 10
7.4 Función `en_franja(nombre)` 10
7.5 Ejemplos 11
8 Carga de datos 12
8.1 El archivo `escuchas.csv` 12
8.2 Funciones de carga (archivo `lectura.py`) 12
9 Pruebas 13
9.1 Qué tiene que incluir cada archivo 13
9.2 Cómo escribir cada prueba 14
9.3 Una mirada a los datos 14
10 Entregable 14
11 Evaluación 15
12 Ayuda 15

1 Introducción

En este trabajo vas a analizar una semana de escuchas de una plataforma de música: qué canción se escuchó, qué día y a qué hora. Los datos vienen en el archivo `escuchas.csv` y, como pasa con

los datos reales, tienen **errores y filas repetidas**. Vas a construir en Python las clases y funciones para cargarlos, limpiarlos, ordenarlos y analizarlos, y al final vas a descubrir qué géneros se escuchan en cada momento del día.

El trabajo integra varios temas de la materia y cada concepto debe aparecer donde hace falta para resolver el problema:

1. **De la Unidad 1:** leer un archivo de texto y agrupar datos con diccionarios.
2. **De la Unidad 2:** diseñar clases con atributos privados y propiedades, sobrecargar operadores y crear una excepción a medida que hereda de `ValueError`.
3. **De la Unidad 3:** usar funciones como argumentos y como resultado (lambdas y un *closure*) y leer el archivo con un generador.
4. **De la Unidad 4:** implementar una cola como lista enlazada para una ventana deslizante y justificar su complejidad.

Vas a programar estas piezas, en este orden:

Orden	Pieza	Archivo	Para qué sirve
1	`DatoInvalidoError`	`excepciones.py`	Avisar que un dato es inválido y en qué campo
2	`Cancion`	`cancion.py`	Una canción con sus datos
3	`Escucha`	`escucha.py`	Un evento: una canción escuchada un día a una hora
4	`Nodo` y `Cola`	`cola.py`	Una cola genérica hecha con nodos enlazados
5	`Historial` y `en_franja`	`historial.py`	Una colección de escuchas con operaciones de análisis
6	`leer_filas` y `cargar_escuchas`	`lectura.py`	Cargar y limpiar el CSV

Te recomendamos seguir ese orden y no pasar a la clase siguiente hasta que la anterior funcione con todos sus ejemplos.

2 Reglas generales

Estas reglas valen para **todo** el trabajo. Si en algún punto tenés una duda, volvé a esta sección.

1. **Librerías permitidas.** Además de tus propios módulos, solo se puede importar un módulo de la librería estándar: `csv`, para leer el archivo (es opcional, ver sección 8.2).

2. **Nombres exactos.** Las clases, métodos, parámetros y atributos deben llamarse exactamente como aparecen en esta consigna.
3. **Errores.** Cuando un dato es inválido, el método solo debe **lanzar una excepción** con `raise`.
4. **Mensajes de error libres.** El texto del mensaje de cada excepción lo elegís vos, pero tiene que explicar qué salió mal. Lo que se evalúa es que el **tipo** de excepción tenga sentido.
5. **Nuevo objeto o mismo objeto.** En la tabla de métodos de `Historial` (sección 7.2), la columna Tipo indica si el método modifica el objeto original (`modifica`) o devuelve un objeto nuevo sin modificar el original (`devuelve nuevo`).
6. **Los objetos no se copian.** Cuando una `Escucha` guarda una canción, o un `Historial` guarda escuchas, guardan **el mismo objeto**, no una copia. Lo que sí se copia es la **lista** que los contiene.
7. **Encapsulamiento.** Los atributos de las clases son privados y se leen a través de propiedades. Desde afuera de una clase no se accede a sus atributos privados: por ejemplo, `Historial` usa la `Cola` solo a través de sus métodos públicos (`encolar`, `desencolar`, `for`, `len()`), sin tocar sus nodos.
8. **Reutilización.** Si un método se puede escribir usando otro que ya hiciste, hacelo así. La consigna marca con ♻ los casos en los que se espera esa reutilización.
9. **Código de prueba.** Los archivos de las clases pueden tener código de prueba solo dentro de `if __name__ == "__main__":`, para que no se ejecute al importarlos.

3 Excepción `DatoInvalidoError` (archivo `excepciones.py`)

Cuando se cargan datos reales, no alcanza con saber que una fila es inválida: hay que saber **por qué**, para poder informarlo. Para eso vas a crear tu propia excepción.

`DatoInvalidoError` **hereda de** `ValueError` y agrega un atributo `campo`, con el nombre del dato que estaba mal (`"hora"`, `"dia"`, `"genero"`, etc.).

Elemento	Descripción
`DatoInvalidoError(campo, mensaje)`	Crea la excepción. `mensaje` es el texto que se muestra al imprimirla (pasalo a `super().__init__`).
`campo`	Atributo con el nombre del campo inválido.

Cada vez que una clase recibe un dato que no puede aceptar, tiene que lanzar una excepción. Parte del trabajo es decidir cuál corresponde en cada caso: `DatoInvalidoError` o alguna de las excepciones propias de Python. Pensá qué tienen en común los casos que van a una u otra: en la presentación vas a tener que justificar tus decisiones.

```
>>> e = DatoInvalidoError("hora", "Hora fuera de rango: 25.")
>>> print(e)
Hora fuera de rango: 25.
>>> e.campo
```

```
'hora'
>>> isinstance(e, ValueError)
True
```

4 Clase `Cancion` (archivo `cancion.py`)

Una `Cancion` guarda los datos de un tema musical: título, artista y género.

4.1 Inicialización: `Cancion(titulo, artista, genero)`

Parámetro	Tipo esperado	Reglas
`titulo`	`str`	No puede estar vacío ni ser solo espacios
`artista`	`str`	Ídem `titulo`
`genero`	`str`	Debe ser uno de `Cancion.GENEROS`

4.2 Atributos y métodos

Nombre	Descripción
`GENEROS`	Puede ser una tupla como `("rock", "pop", "folklore", "cumbia")`. Tener en cuenta que es la misma para todas las canciones.
`titulo`, `artista`, `genero`	Propiedades de solo lectura.
`__repr__(self)`	`Cancion('<titulo>', '<artista>', <genero>)`. Ver ejemplos.
`__eq__(self, other)`	Dos canciones son iguales si tienen el mismo `titulo` y el mismo `artista`, **sin distinguir mayúsculas de minúsculas**. Si `other` no es una `Cancion`, devuelve `False`.

4.3 Ejemplos

```
>>> c = Cancion("Ji ji ji", "Los Redondos", "rock")
>>> c
Cancion('Ji ji ji', 'Los Redondos', rock)
>>> c.genero
'rock'
>>> Cancion.GENEROS
('rock', 'pop', 'folklore', 'cumbia')
>>> c.GENEROS
('rock', 'pop', 'folklore', 'cumbia')
>>> c == Cancion("JI JI JI", "los redondos", "pop")
True
```

```
>>> Cancion("", "X", "rock")
DatoInvalidoError: titulo no puede estar vacío.
>>> Cancion("Hola", "X", "tango")
DatoInvalidoError: Género desconocido: 'tango'.
>>> Cancion("Hola", "   ", "rock")
DatoInvalidoError: artista no puede estar vacío.
>>> Cancion("Hola", 42, "rock")
TypeError: artista debe ser de tipo str.
```

5 Clase `Escucha` (archivo `escucha.py`)

Una `Escucha` es un **evento**: una canción que alguien escuchó un día de la semana a una hora. Cada fila del CSV se convierte en una `Escucha`.

5.1 Atributos de clase

```
DIAS = ("lun", "mar", "mie", "jue", "vie", "sab", "dom")
FRANJAS = {
    "madrugada": (0, 5),
    "mañana": (6, 11),
    "tarde": (12, 17),
    "noche": (18, 23),
}
```

En `FRANJAS`, cada tupla tiene la hora de inicio y la de fin, **ambas incluidas**.

5.2 Inicialización: `Escucha(cancion, dia, hora)`

Parámetro	Reglas
`cancion`	Debe ser una `Cancion`
`dia`	Debe ser uno de `Escucha.DIAS`
`hora`	Un `int` entre 0 y 23, o un `str` formado solo por dígitos, que se convierte a `int` (así llega desde el CSV)

5.3 Propiedades y métodos especiales

Nombre	Descripción
`cancion`, `dia`, `hora`	Propiedades de solo lectura. `hora` siempre es un `int`.
`momento`	Propiedad: la tupla `(posición del día en DIAS, hora)`. Por ejemplo, martes a las 9 → `(1, 9)`. Sirve para comparar escuchas en el tiempo.

Nombre	Descripción
`franja`	Propiedad: el nombre de la franja de `FRANJAS` a la que pertenece la hora.
`__repr__(self)`	`Escucha(<dia> <hora con 2 dígitos>h, '<titulo>', <genero>)`. Ver ejemplos.
`__eq__(self, other)`	Dos escuchas son iguales si tienen la **misma canción** (según `Cancion.__eq__`), el mismo día y la misma hora. Si `other` no es una `Escucha`, devuelve `False`.
`__lt__(self, other)`	`True` si `self` ocurrió antes que `other` (compará sus `momento`). Si `other` no es una `Escucha`, error.

5.4 Ejemplos

En estos ejemplos, `c` es la canción "Ji ji ji" de la sección 4.3.

```
>>> luna = Cancion("Luna tucumana", "Atahualpa Yupanqui", "folklore")
>>> e = Escucha(luna, "lun", 8)
>>> e
Escucha(lun 08h, 'Luna tucumana', folklore)
>>> e.franja
'mañana'
>>> e.momento
(0, 8)
>>> Escucha(c, "sab", "2")
Escucha(sab 02h, 'Ji ji ji', rock)
>>> [Escucha(c, "lun", h).franja for h in (5, 6, 17, 18)]
['madrugada', 'mañana', 'tarde', 'noche']

>>> Escucha(c, "vie", 23) == Escucha(c, "vie", 23)
True
>>> Escucha(c, "vie", 23) == Escucha(c, "vie", 22)
False
>>> Escucha(luna, "lun", 8) < Escucha(c, "mar", 1)
True
>>> sorted([Escucha(c, "vie", 23), Escucha(luna, "lun", 8), Escucha(c, "lun",
21)])
[Escucha(lun 08h, 'Luna tucumana', folklore), Escucha(lun 21h, 'Ji ji ji',
rock), Escucha(vie 23h, 'Ji ji ji', rock)]

>>> Escucha(c, "lunes", 8)
DatoInvalidoError: Día inválido: 'lunes'.
>>> Escucha(c, "lun", 25)
DatoInvalidoError: Hora fuera de rango: 25.
```

```
>>> Escucha(c, "lun", "xx")
DatoInvalidoError: Hora inválida: 'xx'.
>>> Escucha("Ji ji ji", "lun", 8)
TypeError: cancion debe ser un objeto Cancion.
```

6 Clase `Cola` (archivo `cola.py`)

Una **cola** guarda elementos en orden de llegada: el primero que entra es el primero que sale (**FIFO**). Se agrega por el **fondo** y se saca por el **frente**.

En este trabajo la cola se implementa como una **lista enlazada**, no con una lista de Python. ¿Por qué? Con una lista, sacar el primer elemento (`lista.pop(0)`) obliga a correr un lugar todos los demás, y eso es lento cuando hay muchos datos. Con nodos enlazados, sacar del frente es inmediato, sin importar cuántos elementos haya.

6.1 Clase `Nodo`

Un `Nodo` es "una cajita" que guarda un elemento y sabe cuál es el nodo que viene detrás:

Atributo	Descripción
`valor`	El elemento guardado. Se recibe al crear el nodo: `Nodo(valor)`.
`siguiente`	El nodo que viene detrás en la cola, o `None` si es el último. Empieza en `None`.

```
frente                                  fondo
  ↓                                       ↓
[ 1 | siguiente ] → [ 2 | siguiente ] → [ 3 | None ]
```

6.2 Clase `Cola`

La `Cola` guarda tres atributos privados: el nodo del frente (`_frente`), el nodo del fondo (`_fondo`) y la cantidad de elementos (`_cantidad`). **No puede usar una lista de Python** para guardar los elementos. Debe ser **genérica**: tiene que poder guardar cualquier tipo de objeto.

Método	Descripción
`Cola()`	Crea una cola vacía: `_frente` y `_fondo` en `None` y `_cantidad` en 0.
`encolar(self, x)`	Crea un `Nodo` con `x` y lo engancha detrás del fondo (si la cola estaba vacía, el nodo nuevo es frente y fondo a la vez). No devuelve nada.
`desencolar(self)`	Saca el elemento del frente y lo **devuelve**; el frente pasa a ser el nodo siguiente. Si la cola

Método	Descripción
	queda vacía, el fondo también vuelve a `None`. Si ya estaba vacía, error.
`frente(self)`	Devuelve el elemento del frente **sin sacarlo**. Si la cola está vacía, error.
`esta_vacia(self)`	`True` si no tiene elementos.
`__len__(self)`	Cantidad de elementos.
`__iter__(self)`	Recorre los elementos del frente hacia el fondo, siguiendo los nodos, **sin modificar** la cola.

```
>>> cola = Cola()
>>> cola.esta_vacia()
True
>>> cola.encolar(1)
>>> cola.encolar(2)
>>> cola.encolar(3)
>>> list(cola)
[1, 2, 3]
>>> len(cola)
3
>>> cola.frente()
1
>>> cola.desencolar()
1
>>> list(cola)
[2, 3]
>>> len(cola)          # recorrerla no la vació
2
>>> cola.desencolar()
2
>>> cola.desencolar()
3
>>> cola.esta_vacia()
True
>>> cola.desencolar()
IndexError: La cola está vacía.
>>> cola.frente()
IndexError: La cola está vacía.
```

7 Clase `Historial` y función `en_franja` (archivo `historial.py`)

Un `Historial` es una colección de escuchas. Internamente guarda una **lista privada** de objetos `Escucha` (puede tener repetidas).

Sus métodos de análisis reciben **funciones como argumento**, que indican qué mirar de cada escucha. Por ejemplo, `lambda e: e.cancion.genero` agrupa por género y `lambda e: e.franja` agrupa por franja horaria. Es la misma idea que elegir la columna por la que se agrupa en pandas, pero más flexible.

7.1 Datos de ejemplo

Los ejemplos de esta sección usan este historial chico, con una escucha repetida y en desorden. Copialo tal cual en `test_historial.py`.

```
luna = Cancion("Luna tucumana", "Atahualpa Yupanqui", "folklore")
jiji = Cancion("Ji ji ji", "Los Redondos", "rock")
fuiste = Cancion("Fuiste", "Gilda", "cumbia")
milhoras = Cancion("Mil horas", "Los Abuelos de la Nada", "pop")
h = Historial([
    Escucha(jiji, "vie", 23),
    Escucha(luna, "lun", 8),
    Escucha(milhoras, "lun", 15),
    Escucha(fuiste, "sab", 2),
    Escucha(jiji, "lun", 21),
    Escucha(luna, "mar", 9),
    Escucha(jiji, "vie", 23),   # repetida
    Escucha(fuiste, "sab", 1),
])
```

7.2 Métodos

`Historial(escuchas=None)` recibe una **lista** de objetos `Escucha` (si se omite, empieza vacío) y guarda una copia de esa lista.

Método	Tipo	Descripción
`agregar(self, escucha)`	modifica	Agrega `escucha` al final. Si no es una `Escucha`, error.
`filtrar(self, f)`	devuelve nuevo	Las escuchas para las que `f(e)` devuelve `True`.
`sin_duplicados(self)`	devuelve nuevo	Quita las escuchas repetidas (según `Escucha.__eq__`) y conserva la **primera** aparición de cada una.
`ordenado(self)`	devuelve nuevo	Las escuchas en orden cronológico. Si dos ocurrieron en el mismo momento, se mantiene su orden original. ♻️

Método	Tipo	Descripción
		Usá `sorted()`, que funciona gracias a `Escucha.__lt__`.
`conteo_por(self, clave)`	devuelve `dict`	Cuántas escuchas hay por cada valor de `clave(e)`. Ordenado de **mayor a menor cantidad**; si hay empates, primero el valor que aparece antes en el historial.
`tendencias(self, n, clave)`	devuelve `list`	Una **ventana deslizante** de `n` escuchas, hecha con una `Cola` (ver sección 7.3). Devuelve una lista con el valor de `clave` más frecuente en cada ventana. `n` debe ser un `int` mayor o igual a 1: si no es `int`, error; si es menor a 1, error.
`__len__`, `__iter__`, `__getitem__`	—	Cantidad de escuchas, recorrerlas con `for` y acceder a una por posición (`h[0]`, `h[-1]`).
`__repr__(self)`	—	`Historial(escuchas: <cantidad>)`.

7.3 Cómo funciona `tendencias`

Se recorren las escuchas en orden y se mantiene una `Cola` con las últimas `n`:

1. Se encola la escucha actual.
2. Si la cola tiene más de `n` escuchas, se desencola la más vieja.
3. Si la cola tiene exactamente `n` escuchas, se agrega al resultado el valor de `clave` más frecuente en la ventana. Si hay empate, gana el que aparece primero en la ventana, es decir, el más antiguo.

♻ Armá un `Historial` con las escuchas de la cola y usá `conteo_por`.

El resultado tiene `len(self) - n + 1` elementos, o ninguno si `n` es mayor que la cantidad de escuchas. Es lo mismo que hace `rolling()` en pandas, y por eso conviene una cola: en cada paso se saca un elemento del frente.

7.4 Función `en_franja(nombre)`

Es una función que **devuelve otra función** (un *closure*). `en_franja("noche")` devuelve una función que recibe una escucha y devuelve `True` si es de noche, lista para pasar a `filtrar`. Si `nombre` no es una franja de `Escucha.FRANJAS`, error en el momento de llamar a `en_franja`.

7.5 Ejemplos

```
>>> h
Historial(escuchas: 8)
>>> h[0]
Escucha(vie 23h, 'Ji ji ji', rock)
>>> h.sin_duplicados()
Historial(escuchas: 7)
>>> o = h.sin_duplicados().ordenado()
>>> for e in o:
...     print(e)
Escucha(lun 08h, 'Luna tucumana', folklore)
Escucha(lun 15h, 'Mil horas', pop)
Escucha(lun 21h, 'Ji ji ji', rock)
Escucha(mar 09h, 'Luna tucumana', folklore)
Escucha(vie 23h, 'Ji ji ji', rock)
Escucha(sab 01h, 'Fuiste', cumbia)
Escucha(sab 02h, 'Fuiste', cumbia)
>>> h[0]                  # h no cambió
Escucha(vie 23h, 'Ji ji ji', rock)
```

Filtrar, con una lambda o con el closure:

```
>>> list(o.filtrar(lambda e: e.dia == "sab"))
[Escucha(sab 01h, 'Fuiste', cumbia), Escucha(sab 02h, 'Fuiste', cumbia)]
>>> list(o.filtrar(en_franja("mañana")))
[Escucha(lun 08h, 'Luna tucumana', folklore), Escucha(mar 09h, 'Luna
tucumana', folklore)]
>>> noche = en_franja("noche")
>>> noche(o[2])
True
>>> en_franja("siesta")
ValueError: Franja desconocida: 'siesta'.
```

Agregaciones:

```
>>> o.conteo_por(lambda e: e.cancion.genero)
{'folklore': 2, 'rock': 2, 'cumbia': 2, 'pop': 1}
>>> o.conteo_por(lambda e: e.franja)
{'mañana': 2, 'noche': 2, 'madrugada': 2, 'tarde': 1}
```

Tendencias:

```
>>> o.tendencias(3, lambda e: e.cancion.genero)
['folklore', 'pop', 'rock', 'folklore', 'cumbia']
>>> o.tendencias(10, lambda e: e.cancion.genero)
[]
```

```
>>> o.tendencias(0, lambda e: e.franja)
ValueError: n debe ser al menos 1.
```

En el primer ejemplo de `tendencias` con `n = 3`, cada elemento de la lista resultante corresponde a una ventana de 3 escuchas consecutivas de `o` (los datos de ejemplo de la sección 7.1 sin la escucha repetida y ordenados). Como `o` tiene 7 escuchas, hay 5 ventanas. Cuando en una ventana hay empate, gana el género de la escucha más antigua:

Ventana	Escuchas en la ventana (de la más antigua a la más nueva)	Resultado
1	folklore (lun 8h), pop (lun 15h), rock (lun 21h)	`folklore`: empate, gana la más antigua
2	pop (lun 15h), rock (lun 21h), folklore (mar 9h)	`pop`: empate, gana la más antigua
3	rock (lun 21h), folklore (mar 9h), rock (vie 23h)	`rock`: aparece 2 veces
4	folklore (mar 9h), rock (vie 23h), cumbia (sab 1h)	`folklore`: empate, gana la más antigua
5	rock (vie 23h), cumbia (sab 1h), cumbia (sab 2h)	`cumbia`: aparece 2 veces

8 Carga de datos

8.1 El archivo `escuchas.csv`

Antes de seguir, descargá el archivo desde acá.

El archivo tiene un encabezado y una escucha por línea, con sus campos separados por coma (ningún título tiene comas):

```
titulo,artista,genero,dia,hora
Nunca me faltes,Antonio Ríos,cumbia,sab,8
Ji ji ji,Los Redondos,rock,dom,16
```

Las filas están en desorden, algunas están repetidas y otras tienen errores a propósito, como pasa con los datos reales.

8.2 Funciones de carga (archivo `lectura.py`)

Función	Descripción
`leer_filas(ruta)`	Un **generador**: abre el archivo (con `encoding="utf-8"`), saltea el encabezado y

Función	Descripción
	las líneas vacías, y entrega con `yield` cada fila como una lista de textos. Podés separar los campos de dos maneras: con `linea.strip().split(",")` o con `csv.reader`. Las dos son válidas y dan el mismo resultado con este archivo. En ambos casos, el archivo se lee de a una línea, sin cargarlo entero en memoria.
`cargar_escuchas(ruta)`	Devuelve una tupla `(historial, descartes)`. Recorre las filas de `leer_filas` y, con cada una, crea la `Cancion` y la `Escucha` y la agrega al historial. Si una fila no tiene 5 campos, o si al crear los objetos se lanza `DatoInvalidoError`, la fila se **descarta** y se suma 1 en `descartes[campo]`. Contá las filas con una cantidad incorrecta de campos con la clave `"formato"`. Al final, `descartes` dice cuántas filas se descartaron por cada motivo, por ejemplo `{"hora": 2, "formato": 1}`. Las demás excepciones no se atrapan.

Para que puedas verificar tu carga: con el archivo entregado, deberían quedar **303 escuchas válidas** y **12 filas descartadas**.

9 Pruebas

Las pruebas son dos archivos que, al ejecutarse, muestran en pantalla que las clases funcionan. Cada uno tiene que correr **de principio a fin sin cortarse**, con `python test_clases.py` y `python test_historial.py`.

9.1 Qué tiene que incluir cada archivo

Archivo	Debe incluir todos los ejemplos de
`test_clases.py`	Secciones 3 a 5 (`DatoInvalidoError`, `Cancion` y `Escucha`)
`test_historial.py`	Sección 6 (`Cola`), sección 7 (`Historial` y `en_franja`), usando los datos de la sección 7.1, y sección 9.3

Además de los ejemplos de la consigna, cada archivo debe tener **al menos 2 pruebas propias** (inventadas por vos) que muestren algún caso no cubierto por los ejemplos. Marcalas con el comentario `# Prueba propia`.

9.2 Cómo escribir cada prueba

1. Antes de cada prueba, un **comentario** que diga qué se prueba.
2. Mostrá el resultado con `print`, junto con un texto que permita entenderlo.
3. Las pruebas que **deben lanzar un error** se escriben dentro de `try`/`except`, para que el archivo siga ejecutándose. Si el error no aparece, la prueba tiene que avisarlo.
4. Cada `test_*.py` crea sus propios datos: no depende de que otro archivo se haya ejecutado antes.

Modelo a seguir:

```
# La franja de una escucha a las 8 es "mañana"
print("Franja de las 8 h:", Escucha(luna, "lun", 8).franja)

# Una hora fuera de rango debe lanzar un error
try:
    Escucha(luna, "lun", 25)
    print("ERROR: Escucha(luna, 'lun', 25) no lanzó ninguna excepción")
except ValueError as e:
    print("OK, ValueError:", e)
```

Salida esperada:

```
Franja de las 8 h: mañana
OK, ValueError: Hora fuera de rango: 25.
```

9.3 Una mirada a los datos

Al final de `test_historial.py`, cargá `escuchas.csv` con `cargar_escuchas`, eliminá los duplicados y mostrá el género dominante de cada franja horaria (usá `en_franja` y `conteo_por`). Vas a ver que el archivo esconde un patrón.

10 Entregable

La entrega consiste **exclusivamente** en un archivo comprimido `.zip` con estos 8 archivos, con estos nombres exactos y sin carpetas adentro:

Archivo	Contenido
`excepciones.py`	Excepción `DatoInvalidoError`
`cancion.py`	Clase `Cancion`
`escucha.py`	Clase `Escucha`
`cola.py`	Clases `Nodo` y `Cola`
`historial.py`	Clase `Historial` y función `en_franja`
`lectura.py`	Funciones `leer_filas` y `cargar_escuchas`

Archivo	Contenido
`test_clases.py`	Pruebas de las secciones 3 a 5
`test_historial.py`	Pruebas de las secciones 6, 7 y 9.3

No hace falta incluir `escuchas.csv`: se corrige con el archivo original.

No se deben agregar otros archivos ni usar otros nombres. El archivo comprimido se llama `{Apellido}_{Nombre}.zip`, por ejemplo `Alvarez_Julian.zip`.

11 Evaluación

La evaluación tiene dos partes.

1. **Ejecución y revisión del código.** Se verifica que los dos archivos de prueba se ejecuten completos y sin errores, y que las clases cumplan con esta consigna. También se tienen en cuenta la claridad y organización del código, los comentarios, las validaciones y la reutilización de métodos (los casos marcados con ♻). Esta parte se califica de 0 a 10 y se aprueba con 6. Quienes la aprueben pasan a la presentación individual; quienes no, pueden rendir el examen final en condición regular.
2. **Presentación individual.** Hasta 10 minutos para explicar cómo implementaste las clases y mostrar las pruebas funcionando. Luego, hasta 10 minutos de preguntas sobre tu código.

Nota importante. Los archivos de prueba deberían incluir todos los ejemplos de la consigna. Se consideran válidas las entregas que incluyan **al menos el 80 % de los ejemplos**, siempre que todos los que estén se ejecuten correctamente. Las 2 pruebas propias por archivo son obligatorias.

12 Ayuda

- **Orden de trabajo.** Seguí el orden de la tabla de la sección 1. Cada pieza usa las anteriores, así que un error en `Cancion` aparece después en el historial.
- **`isinstance` y `bool`.** Como `isinstance(True, int)` da `True`, para aceptar solo enteros hay que pedir `isinstance(x, int) and not isinstance(x, bool)`.
- **Excepción a medida.** En el `__init__` de `DatoInvalidoError`, llamá a `super().__init__(mensaje)` para que el mensaje se muestre al imprimirla, y después guardá `campo`.
- **Leer un archivo.** `with open(ruta, encoding="utf-8") as archivo:` abre el archivo y lo cierra solo. `next(archivo)` saltea una línea (el encabezado) y `for linea in archivo:` recorre las demás.
- **El módulo `csv`.** Si preferís usarlo, abrí el archivo con `open(ruta, encoding="utf-8", newline="")` y creá un lector con `lector = csv.reader(archivo)`. El lector se recorre y entrega cada fila ya separada en una lista de textos; `next(lector)` saltea el encabezado. Ojo: las líneas vacías llegan como listas vacías (`[]`), así que hay que saltearlas igual. Como `csv.reader` también lee de a una fila, `leer_filas` sigue siendo un generador.

- **Ordenar un diccionario.** `dict(sorted(d.items(), key=lambda par: par[1], reverse=True))` arma un diccionario nuevo ordenado por los valores, de mayor a menor. Mantiene el orden original en los empates.
- **Lista enlazada.** Antes de programar `encolar` y `desencolar`, podés dibujar en papel los nodos y las flechas para tres casos: la cola vacía, con un solo elemento y con varios. La mayoría de los errores aparecen en los dos primeros.
- **Formato con ceros.** `f"{7:02d}"` da `'07'`: `:02d` significa "mostrá el entero con al menos 2 dígitos, completando con ceros a la izquierda" (sin ese formato, `f"{7}"` da `'7'`). Te sirve para el `__repr__` de `Escucha`: así, una escucha a las 8 se muestra como `08h`.
- **Reutilizá.** Antes de escribir un método, preguntate si se puede armar con otros que ya tengas. Por ejemplo, `tendencias` usa `conteo_por` para contar cada ventana, y el género dominante de una franja sale de combinar `filtrar` con `conteo_por`.